



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 02

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 21 de febrero del 2026

CALIFICACIÓN: _____

ACTIVIDADES REALIZADAS

Generación de figuras

Para el primer ejercicio de clase se pidió generar distintas figuras copiando el código original que generaba el triángulo rojo y el cuadrado verde, esto se logro gracias la copia de los códigos dentro de la función `CrearLetrasyFiguras()` como se muestra a continuación.

```
94     void CrearLetrasyFiguras()
95     {
96         //Triangulo diagonal Azul
97         > GLfloat vertices_letras[] = { ... }
106         MeshColor *letras = new MeshColor();
107         letras->CreateMeshColor(vertices_letras,18);
108         meshColorList.push_back(letras);
109
110         // -----
111         // TRIANGULOS RGB [1,2,3] -----
112         GLfloat vertices_trianguloROJO[] = {
113             //X      Y      Z      R      G      B
114             -1.0f,  -1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
115             1.0f,   -1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
116             0.0f,   1.0f,   0.5f,   1.0f,   0.0f,   0.0f,
117         };
118         MeshColor* trianguloROJO = new MeshColor();
119         trianguloROJO->CreateMeshColor(vertices_trianguloROJO, 18);
120         meshColorList.push_back(trianguloROJO);
121
122         GLfloat vertices_trianguloVERDE[] = {
123             //X      Y      Z      R      G      B
124             -1.0f,  -1.0f,   0.5f,   0.224f, 0.310f, 0.208f,
125             1.0f,   -1.0f,   0.5f,   0.224f, 0.310f, 0.208f,
126             0.0f,   1.0f,   0.5f,   0.224f, 0.310f, 0.208f,
127         };
128         MeshColor* trianguloVERDE = new MeshColor();
129         trianguloVERDE->CreateMeshColor(vertices_trianguloVERDE, 18);
130         meshColorList.push_back(trianguloVERDE);
131
132         ~ GLfloat vertices_trianguloAZUL[] = {
133             //X      Y      Z      R      G      B
134             -1.0f,  -1.0f,   0.5f,   0.137f, 0.412f, 0.510f,
135             1.0f,   -1.0f,   0.5f,   0.137f, 0.412f, 0.510f,
136             0.0f,   1.0f,   0.5f,   0.137f, 0.412f, 0.510f,
137         };
138         MeshColor* trianguloAZUL = new MeshColor();
139         trianguloAZUL->CreateMeshColor(vertices_trianguloAZUL, 18);
140         meshColorList.push_back(trianguloAZUL);
141     }
```

El único cambio que se realizó fue en el RGB y el renombrar las listas, esto mismo se repitió para la creación de los cuadrados rojo, azul y café.

```

142 // -----
143 // CUADRADOS RG-CAFE [4,5,6] -----
144 GLfloat vertices_cuadradoROJO[] = {
145     //X      Y      Z      R      G      B
146     -0.5f,  -0.5f,   0.5f,   0.620f, 0.290f, 0.302f,
147     0.5f,   -0.5f,   0.5f,   0.620f, 0.290f, 0.302f,
148     0.5f,    0.5f,   0.5f,   0.620f, 0.290f, 0.302f,
149     -0.5f,  -0.5f,   0.5f,   0.620f, 0.290f, 0.302f,
150     0.5f,    0.5f,   0.5f,   0.620f, 0.290f, 0.302f,
151     -0.5f,    0.5f,   0.5f,   0.620f, 0.290f, 0.302f,
152 };
153 MeshColor* cuadradoROJO = new MeshColor();
154 cuadradoROJO->CreateMeshColor(vertices_cuadradoROJO, 36);
155 meshColorList.push_back(cuadradoROJO);
156
157 GLfloat vertices_cuadradoVERDE[] = {
158     //X      Y      Z      R      G      B
159     -0.5f,  -0.5f,   0.5f,   0.671f, 0.800f, 0.647f,
160     0.5f,   -0.5f,   0.5f,   0.671f, 0.800f, 0.647f,
161     0.5f,    0.5f,   0.5f,   0.671f, 0.800f, 0.647f,
162     -0.5f,  -0.5f,   0.5f,   0.671f, 0.800f, 0.647f,
163     0.5f,    0.5f,   0.5f,   0.671f, 0.800f, 0.647f,
164     -0.5f,    0.5f,   0.5f,   0.671f, 0.800f, 0.647f,
165 };
166 MeshColor* cuadradoVERDE = new MeshColor();
167 cuadradoVERDE->CreateMeshColor(vertices_cuadradoVERDE, 36);
168 meshColorList.push_back(cuadradoVERDE);
169
170 GLfloat vertices_cuadradoCAFE[] = {
171     //X      Y      Z      R      G      B
172     -0.5f,  -0.5f,   0.5f,   0.369f, 0.286f, 0.227f,
173     0.5f,   -0.5f,   0.5f,   0.369f, 0.286f, 0.227f,
174     0.5f,    0.5f,   0.5f,   0.369f, 0.286f, 0.227f,
175     -0.5f,  -0.5f,   0.5f,   0.369f, 0.286f, 0.227f,
176     0.5f,    0.5f,   0.5f,   0.369f, 0.286f, 0.227f,
177     -0.5f,    0.5f,   0.5f,   0.369f, 0.286f, 0.227f,
178 };
179 MeshColor* cuadradoCAFE = new MeshColor();
180 cuadradoCAFE->CreateMeshColor(vertices_cuadradoCAFE, 36);
181 meshColorList.push_back(cuadradoCAFE);
182 }

```

Se quiso mostrar todas las figuras en pantalla a la vez, así que se reorganizó donde se colocaban mediante la función `translate`. Todo esto dentro de nuestro `main` principal y dentro de nuestra función `while` que se repite cada ciclo de reloj.

Es importante mencionar que para no necesitar la función de `scale` por el momento se quedó la instrucción `glm::mat4 projection = glm::perspective(...);` y se dejó a la función ortogonal comentada.

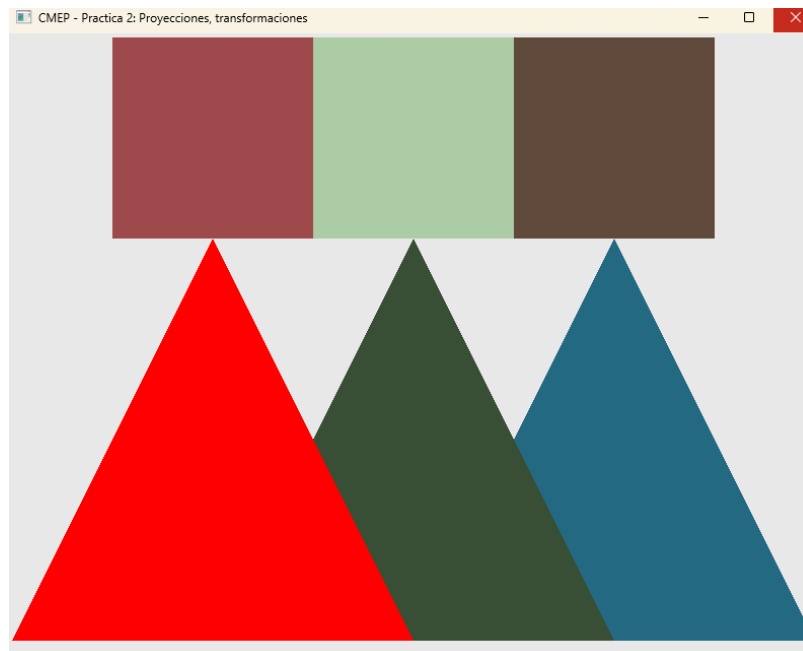
```

229 // -----
230 // DEMOSTRACIÓN DE TODAS LAS FIGURAS -----
231 // -----
232 // TRIANGULO ROJO
233 model = glm::mat4(1.0);
234 model = glm::translate(model, glm::vec3(-1.0f, 0.0f, -4.0f));
235 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
236 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
237 meshColorList[1]->RenderMeshColor();
238 // TRIANGULO VERDE
239 model = glm::mat4(1.0);
240 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
241 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
242 meshColorList[2]->RenderMeshColor();
243 // TRIANGULO AZUL
244 model = glm::mat4(1.0);
245 model = glm::translate(model, glm::vec3(1.0f, 0.0f, -4.0f));
246 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
247 meshColorList[3]->RenderMeshColor();
248 // CUADRADO ROJO
249 model = glm::mat4(1.0);
250 model = glm::translate(model, glm::vec3(-1.0f, 1.5f, -4.0f));
251 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
252 meshColorList[4]->RenderMeshColor();
253 // CUADRADO VERDE
254 model = glm::mat4(1.0);
255 model = glm::translate(model, glm::vec3(0.0f, 1.5f, -4.0f));
256 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
257 meshColorList[5]->RenderMeshColor();
258 // CUADRADO CAFE
259 model = glm::mat4(1.0);
260 model = glm::translate(model, glm::vec3(1.0f, 1.5f, -4.0f));
261 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
262 meshColorList[6]->RenderMeshColor();

```

EJECUCIÓN

Todas las figuras alejadas 4 unidades de la pantalla para verlas mejor



Dibujo de casa

Para el segundo ejercicio se pidió recrear el dibujo de una casa a partir de instancias de las figuras anteriormente creadas, respetando que todas estas se crearan en el origen y que su movimiento fuera mediante transformaciones geométricas.

Se comento entonces las instancias anteriores y se crearon nuevas, en total **1 cuadrado rojo**, **3 cuadrados verdes**, **2 cuadrados cafés**, **1 triángulo azul** y **2 triángulos verdes** (9 instancias nuevas). En esta ocasión si se comentó `glm::mat4 projection = glm::perspective(...);` y se des comentó la proyección en ortogonal `glm::mat4 projection = glm::ortho(...);`

Obteniendo por resultado este código dentro del main y dentro de nuestro ciclo while que se repite cada ciclo de reloj hasta que se cierre la ventana.

```
267 // DIBUJO DE CASITA -----
268 // -----
269 // CUADRADO ROJO
270 model = glm::mat4(1.0);
271 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.2f));
272 model = glm::scale(model, glm::vec3(0.9f, 1.0f, 1.0f));
273 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
274 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
275 meshColorList[4]->RenderMeshColor();
276 // TRIANGULO AZUL
277 model = glm::mat4(1.0);
278 model = glm::translate(model, glm::vec3(0.0f, 0.72f, -2.2f));
279 model = glm::scale(model, glm::vec3(0.5f, 0.22f, 1.0f));
280 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
281 meshColorList[3]->RenderMeshColor();
282 // CUADRADO VERDE 1
283 model = glm::mat4(1.0);
284 model = glm::translate(model, glm::vec3(0.2f, 0.1f, -2.1f));
285 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 1.0f));
286 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
287 meshColorList[5]->RenderMeshColor();
288 // CUADRADO VERDE 2
289 model = glm::mat4(1.0);
290 model = glm::translate(model, glm::vec3(-0.2f, 0.1f, -2.1f));
291 model = glm::scale(model, glm::vec3(0.25f, 0.25f, 1.0f));
292 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
293 meshColorList[5]->RenderMeshColor();
294 // CUADRADO VERDE 3
295 model = glm::mat4(1.0);
296 model = glm::translate(model, glm::vec3(0.0f, -0.33f, -2.1f));
297 model = glm::scale(model, glm::vec3(0.25f, 0.35f, 1.0f));
298 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
299 meshColorList[5]->RenderMeshColor();
300 // CUADRADO CAFE 1
301 model = glm::mat4(1.0);
302 model = glm::translate(model, glm::vec3(-0.68f, -0.395f, -2.1f));
303 model = glm::scale(model, glm::vec3(0.15f, 0.20f, 1.0f));
304 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
305 meshColorList[6]->RenderMeshColor();
306 // CUADRADO CAFE 2
307 model = glm::mat4(1.0);
308 model = glm::translate(model, glm::vec3(0.68f, -0.395f, -2.1f));
309 model = glm::scale(model, glm::vec3(0.15f, 0.20f, 1.0f));
310 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
311 meshColorList[6]->RenderMeshColor();
```

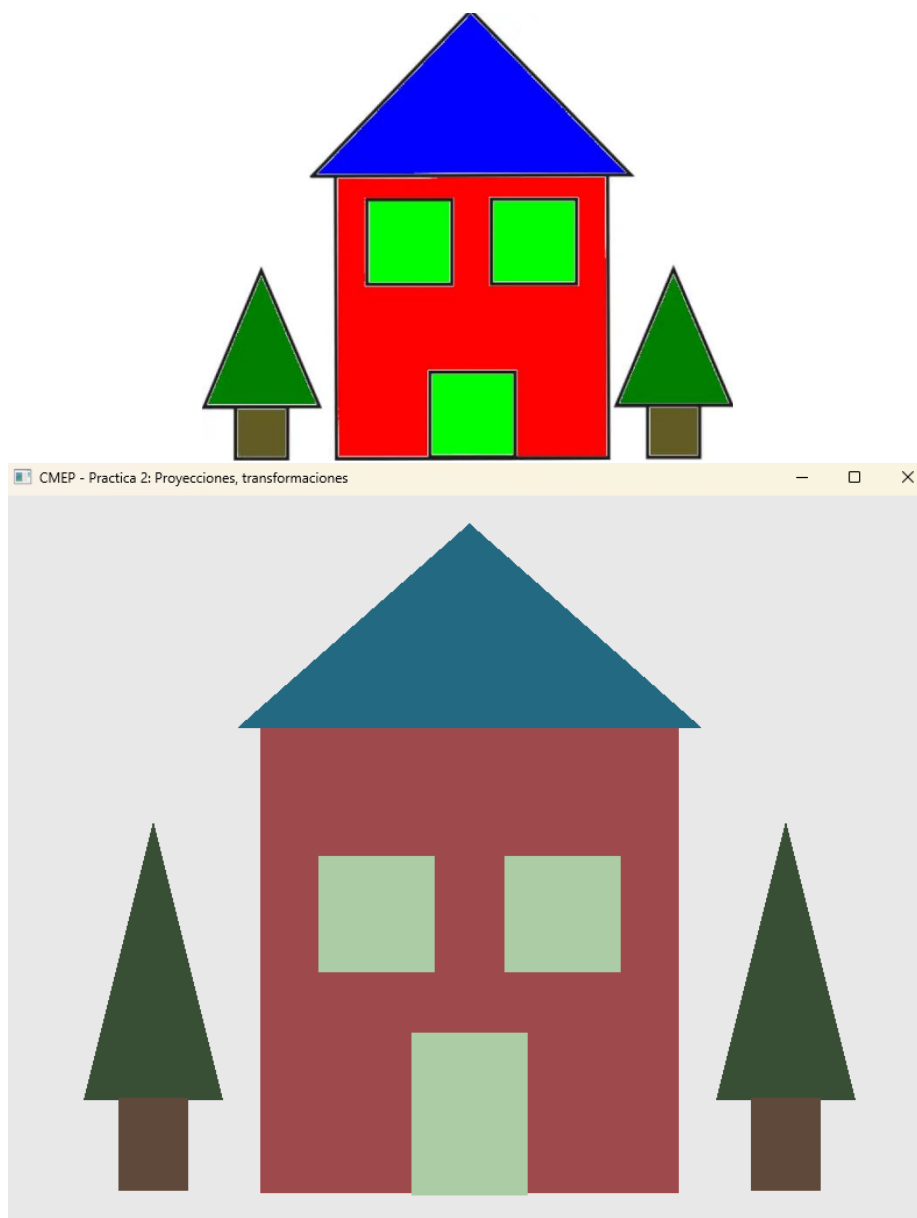
```

312 // TRIANGULO VERDE 1
313 model = glm::mat4(1.0);
314 model = glm::translate(model, glm::vec3(0.68f, 0.0f, -2.1f));
315 model = glm::scale(model, glm::vec3(0.15f, 0.3f, 1.0f));
316 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
317 meshColorList[2]->RenderMeshColor();
318 // TRIANGULO VERDE 2
319 model = glm::mat4(1.0);
320 model = glm::translate(model, glm::vec3(-0.68f, 0.0f, -2.1f));
321 model = glm::scale(model, glm::vec3(0.15f, 0.3f, 1.0f));
322 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
323 meshColorList[2]->RenderMeshColor();

```

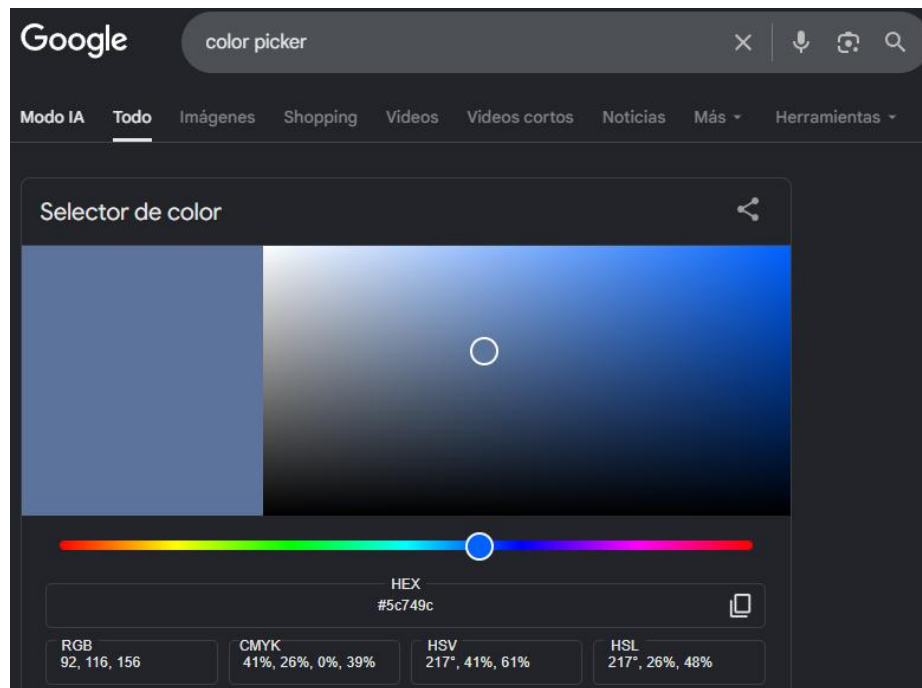
EJECUCIÓN

El fondo se cambió de negro a un gris claro, se adjunta la imagen de referencia que se pidió.



PROBLEMAS PRESENTADOS

No hubo algún problema para esta práctica, sin embargo, los colores pedidos eran demasiado saturados, así que cambiaron por algunas variantes que fueran un poco más grisáceas para una mejor combinación en el color. Para ello se extrajo su código RGB de la función color picker que ofrece Google y el resultado se dividió entre 255.



CONCLUSIONES

Sobre los ejercicios de clase

Hubo algunas dificultades para manejar de manera correcta las transformaciones que se vieron en esta práctica, se estuvo experimentando sobre los resultados de trasladar primero una figura y luego rotarla, en comparación de rotarla primero y luego trasladarla. Además, algunas cosas como el manejo de números en estas funciones siguieron siendo algo que fue tardado y hasta cierto punto complicado si de escalar se habla, mientras que las traslaciones de las figuras para la segunda actividad también llevaron más tiempo del que se estimaba al estar ejecutando a prueba y error para ver que todo quedará lo más fiel al dibujo original.

Comentarios generales

En el laboratorio la explicación fue clara y bien detallada, sin embargo, llego un momento en que seguir el ritmo de la clase fue pesado, entre el tiempo que queríamos intentar lo mismo que hacía el profesor y los momentos en que veíamos nuestra pantalla, uno podía perderse parte del proceso. Creo que sería más eficiente decir el numero de línea que se manipula por si nuestra vista tiene que estar en nuestro equipo al momento de querer seguir los mismos ejemplos que hace el profesor.